MDTS 4214 Assignment 8

Subhayan Biswas

March 30, 2026

1 Problem to demonstrate the fitting of a binary response variable using logistic regression

a. Consider the Weekly data available in the ISLR package of R. Use the data from 1990 to 2008 as train data and the data on 2009-2010 as test data.

[22]:
```
#!pip install ISLP
```

[4]:
```
import numpy as np
import pandas as pd
import statsmodels.api as sm
import matplotlib.pyplot as plt
from sklearn.metrics import roc_curve, confusion_matrix
from ISLP import load_data
```

[5]:
```
df = load_data('Weekly')
df
```

[5]:

	Year	Lag1	Lag2	Lag3	Lag4	Lag5	Volume	Today	Direction
0	1990	0.816	1.572	-3.936	-0.229	-3.484	0.154976	-0.270	Down
1	1990	-0.270	0.816	1.572	-3.936	-0.229	0.148574	-2.576	Down
2	1990	-2.576	-0.270	0.816	1.572	-3.936	0.159837	3.514	Up
3	1990	3.514	-2.576	-0.270	0.816	1.572	0.161630	0.712	Up
4	1990	0.712	3.514	-2.576	-0.270	0.816	0.153728	1.178	Up
…	…	…	…	…	…	…	…	…	…
1084	2010	-0.861	0.043	-2.173	3.599	0.015	3.205160	2.969	Up
1085	2010	2.969	-0.861	0.043	-2.173	3.599	4.242568	1.281	Up
1086	2010	1.281	2.969	-0.861	0.043	-2.173	4.835082	0.283	Up
1087	2010	0.283	1.281	2.969	-0.861	0.043	4.454044	1.034	Up
1088	2010	1.034	0.283	1.281	2.969	-0.861	2.707105	0.069	Up

[1089 rows x 9 columns]

[6]:
```
X_train = df[df['Year'] <= 2008].drop(columns = ['Year', 'Direction', 'Today'])
X_train
```

[6]:

	Lag1	Lag2	Lag3	Lag4	Lag5	Volume
0	0.816	1.572	-3.936	-0.229	-3.484	0.154976

```
1    -0.270   0.816   1.572  -3.936  -0.229  0.148574
2    -2.576  -0.270   0.816   1.572  -3.936  0.159837
3     3.514  -2.576  -0.270   0.816   1.572  0.161630
4     0.712   3.514  -2.576  -0.270   0.816  0.153728
..      …       …      …       …       …       …
980  12.026  -8.389  -6.198  -3.898  10.491  5.841565
981  -2.251  12.026  -8.389  -6.198  -3.898  6.093950
982   0.418  -2.251  12.026  -8.389  -6.198  5.932454
983   0.926   0.418  -2.251  12.026  -8.389  5.855972
984  -1.698   0.926   0.418  -2.251  12.026  3.087105

[985 rows x 6 columns]
```

[7]:
```
X_test = df[df['Year'] > 2008].drop(columns = ['Year', 'Direction', 'Today'])
```

[8]:
```
Y_train = df[df['Year'] <= 2008]['Direction'].map({'Down': 0, 'Up': 1})
Y_test = df[df['Year'] > 2008]['Direction'].map({'Down': 0, 'Up': 1})
```

b. Using the training data, fit a logistic regression with Direction as the response and the five lag variables plus Volume as predictors. Are there any significant predictors? If not, which of the tests is reporting the least p value. Interpret the coefficient corresponding to that test.

[9]:
```
X_train_const = sm.add_constant(X_train)
logit = sm.Logit(Y_train, X_train_const).fit()
logit.summary()
```

```
Optimization terminated successfully.
         Current function value: 0.681388
         Iterations 4
```

[9]:

Dep. Variable:	Direction	**No. Observations:**	985
Model:	Logit	**Df Residuals:**	978
Method:	MLE	**Df Model:**	6
Date:	Mon, 30 Mar 2026	**Pseudo R-squ.:**	0.009136
Time:	15:46:39	**Log-Likelihood:**	-671.17
converged:	True	**LL-Null:**	-677.35
Covariance Type:	nonrobust	**LLR p-value:**	0.05408

	coef	**std err**	**z**	**P> |z|**	**[0.025**	**0.975]**
const	0.3326	0.094	3.530	0.000	0.148	0.517
Lag1	-0.0623	0.029	-2.123	0.034	-0.120	-0.005
Lag2	0.0447	0.030	1.499	0.134	-0.014	0.103
Lag3	-0.0155	0.029	-0.524	0.600	-0.073	0.042
Lag4	-0.0311	0.029	-1.064	0.287	-0.088	0.026
Lag5	-0.0377	0.029	-1.291	0.197	-0.095	0.020
Volume	-0.0897	0.054	-1.658	0.097	-0.196	0.016

Significant Predictors: Looking at the P>|z| column, *Lag*1 is the only statistically significant predictor at the standard 0.05 level, with a p-value of 0.034. *Volume* is marginally significant

(0.097) but generally considered non-significant at the 5% level.

Coefficient Interpretation: The coefficient for $Lag1$ is -0.0623. Because it is negative, this implies that a positive return from the previous week ($Lag1$) actually slightly decreases the log-odds (and therefore the probability) of the market going **"Up"** this week.

c. Estimate the odds of market going "up" at median values of the predictors.

[10]:
```
median = X_train.median()
medianc = [1] + median.tolist()
medianc
```

[10]:
```
[1, 0.231, 0.2339999999999999, 0.231, 0.23, 0.23, 0.804848]
```

[11]:
```
phat = logit.predict(medianc)[0]
odds = phat / (1 - phat)

print(f"Predicted Probability: {phat}")
print(f"Estimated Odds: {odds}")
```

```
Predicted Probability: 0.5589808385120305
Estimated Odds: 1.2674751741535812
```

At the median values of all predictors, the model predicts a 55.9% probability (0.5589) of the market going **"Up"**.

The estimated odds are 1.267. This means that when the predictors are at their median values, the market is roughly 1.26 times more likely to go *Up* than to go *Down*.

e. Using the fitted model and the predictor values from the test data, find an optimal cut point that minimizes $TPR(1 - FPR)$. Report the confusion matrix corresponding to the optimal cut-point value. From the confusion matrix compute the test error rate.

[21]:
```
y_train_pred_prob = logit.predict(X_train_const)
fpr, tpr, thresholds = roc_curve(Y_train, y_train_pred_prob)
print(fpr, tpr)
```

```
[0.         0.00226757 0.00226757 0.00453515 0.00453515 0.00680272
 0.00680272 0.00907029 0.00907029 0.01133787 0.01133787 0.01360544
 0.01360544 0.01814059 0.01814059 0.02267574 0.02267574 0.02494331
 0.02494331 0.02721088 0.02721088 0.03401361 0.03401361 0.03628118
 0.03628118 0.03854875 0.03854875 0.04081633 0.04081633 0.04535147
 0.04535147 0.04761905 0.04761905 0.04988662 0.04988662 0.0521542
 0.0521542  0.05668934 0.05668934 0.06122449 0.06122449 0.06349206
 0.06349206 0.07029478 0.07029478 0.08163265 0.08163265 0.08390023
 0.08390023 0.0861678  0.0861678  0.08843537 0.08843537 0.09070295
 0.09070295 0.0952381  0.0952381  0.09750567 0.09750567 0.09977324
 0.09977324 0.10430839 0.10430839 0.10657596 0.10657596 0.10884354
 0.10884354 0.11111111 0.11111111 0.11337868 0.11337868 0.12244898
 0.12244898 0.12471655 0.12471655 0.12698413 0.12698413 0.13378685
 0.13378685 0.13605442 0.13605442 0.138322   0.138322   0.14058957
```

0.14058957 0.14285714 0.14285714 0.14512472 0.14512472 0.14739229
0.14739229 0.14965986 0.14965986 0.15192744 0.15192744 0.16099773
0.16099773 0.16326531 0.16326531 0.16553288 0.16553288 0.16780045
0.16780045 0.1723356 0.1723356 0.17687075 0.17687075 0.17913832
0.17913832 0.18367347 0.18367347 0.18594104 0.18594104 0.18820862
0.18820862 0.19047619 0.19047619 0.19274376 0.19274376 0.20861678
0.20861678 0.21088435 0.21088435 0.2154195 0.2154195 0.21995465
0.21995465 0.2244898 0.2244898 0.22902494 0.22902494 0.23129252
0.23129252 0.23356009 0.23356009 0.23582766 0.23582766 0.24263039
0.24263039 0.25170068 0.25170068 0.26530612 0.26530612 0.2675737
0.2675737 0.26984127 0.26984127 0.27210884 0.27210884 0.27664399
0.27664399 0.27891156 0.27891156 0.28117914 0.28117914 0.28798186
0.28798186 0.29024943 0.29024943 0.29478458 0.29478458 0.3015873
0.3015873 0.30612245 0.30612245 0.3106576 0.3106576 0.31519274
0.31519274 0.31972789 0.31972789 0.32199546 0.32199546 0.32426304
0.32426304 0.32653061 0.32653061 0.33106576 0.33106576 0.33560091
0.33560091 0.33786848 0.33786848 0.34920635 0.34920635 0.35600907
0.35600907 0.35827664 0.35827664 0.36054422 0.36054422 0.36281179
0.36281179 0.36507937 0.36507937 0.36734694 0.36734694 0.36961451
0.36961451 0.37641723 0.37641723 0.38095238 0.38095238 0.38321995
0.38321995 0.38548753 0.38548753 0.3877551 0.3877551 0.39229025
0.39229025 0.3968254 0.3968254 0.39909297 0.39909297 0.40136054
0.40136054 0.40362812 0.40362812 0.41723356 0.41723356 0.41950113
0.41950113 0.42176871 0.42176871 0.42403628 0.42403628 0.42630385
0.42630385 0.43310658 0.43310658 0.43537415 0.43537415 0.4399093
0.4399093 0.44217687 0.44217687 0.44444444 0.44444444 0.44671202
0.44671202 0.45124717 0.45124717 0.45351474 0.45351474 0.45804989
0.45804989 0.46258503 0.46258503 0.46712018 0.46712018 0.46938776
0.46938776 0.4739229 0.4739229 0.47845805 0.47845805 0.48072562
0.48072562 0.48526077 0.48526077 0.49206349 0.49206349 0.49433107
0.49433107 0.49659864 0.49659864 0.49886621 0.49886621 0.50113379
0.50113379 0.50566893 0.50566893 0.50793651 0.50793651 0.51927438
0.51927438 0.52380952 0.52380952 0.53061224 0.53061224 0.53514739
0.53514739 0.53968254 0.53968254 0.54648526 0.54648526 0.55102041
0.55102041 0.55328798 0.55328798 0.55555556 0.55555556 0.5600907
0.5600907 0.57142857 0.57142857 0.57823129 0.57823129 0.58049887
0.58049887 0.58730159 0.58730159 0.58956916 0.58956916 0.59183673
0.59183673 0.59410431 0.59410431 0.59863946 0.59863946 0.60544218
0.60544218 0.60770975 0.60770975 0.60997732 0.60997732 0.6122449
0.6122449 0.61451247 0.61451247 0.63038549 0.63038549 0.63945578
0.63945578 0.64399093 0.64399093 0.65079365 0.65079365 0.65306122
0.65306122 0.6553288 0.6553288 0.65759637 0.65759637 0.66439909
0.66439909 0.67346939 0.67346939 0.67573696 0.67573696 0.67800454
0.67800454 0.68253968 0.68253968 0.68480726 0.68480726 0.6893424
0.6893424 0.69160998 0.69160998 0.69387755 0.69387755 0.70294785
0.70294785 0.72108844 0.72108844 0.72335601 0.72335601 0.72562358
0.72562358 0.72789116 0.72789116 0.73922902 0.73922902 0.74603175
0.74603175 0.74829932 0.74829932 0.75283447 0.75283447 0.75510204

```
 0.75510204 0.75736961 0.75736961 0.75963719 0.75963719 0.76190476
 0.76190476 0.76417234 0.76417234 0.77097506 0.77097506 0.77777778
 0.77777778 0.78004535 0.78004535 0.7845805  0.7845805  0.78684807
 0.78684807 0.78911565 0.78911565 0.79365079 0.79365079 0.79818594
 0.79818594 0.80498866 0.80498866 0.80725624 0.80725624 0.81179138
 0.81179138 0.82086168 0.82086168 0.82539683 0.82539683 0.8276644
 0.8276644  0.82993197 0.82993197 0.83219955 0.83219955 0.83900227
 0.83900227 0.84126984 0.84126984 0.84353741 0.84353741 0.84580499
 0.84580499 0.85034014 0.85034014 0.86394558 0.86394558 0.86621315
 0.86621315 0.8707483  0.8707483  0.87981859 0.87981859 0.88208617
 0.88208617 0.89342404 0.89342404 0.89795918 0.89795918 0.90022676
 0.90022676 0.90249433 0.90249433 0.90702948 0.90702948 0.90929705
 0.90929705 0.91609977 0.91609977 0.91836735 0.91836735 0.92063492
 0.92063492 0.92517007 0.92517007 0.92970522 0.92970522 0.93424036
 0.93424036 0.93877551 0.93877551 0.94557823 0.94557823 0.9478458
 0.9478458  0.96598639 0.96598639 0.97732426 0.97732426 0.98412698
 0.98412698 0.99319728 0.99319728 0.99546485 0.99546485 1.        ] [0.
0.         0.00735294 0.00735294 0.01102941 0.01102941
 0.01286765 0.01286765 0.01470588 0.01470588 0.01838235 0.01838235
 0.02389706 0.02389706 0.02757353 0.02757353 0.03125    0.03125
 0.03860294 0.03860294 0.04227941 0.04227941 0.05330882 0.05330882
 0.05514706 0.05514706 0.05882353 0.05882353 0.06066176 0.06066176
 0.06433824 0.06433824 0.06617647 0.06617647 0.08088235 0.08088235
 0.08272059 0.08272059 0.08639706 0.08639706 0.09007353 0.09007353
 0.10477941 0.10477941 0.11213235 0.11213235 0.11580882 0.11580882
 0.12132353 0.12132353 0.12683824 0.12683824 0.13051471 0.13051471
 0.13786765 0.13786765 0.14154412 0.14154412 0.14522059 0.14522059
 0.15257353 0.15257353 0.15625    0.15625    0.16176471 0.16176471
 0.16727941 0.16727941 0.17279412 0.17279412 0.17830882 0.17830882
 0.18014706 0.18014706 0.18198529 0.18198529 0.18382353 0.18382353
 0.19301471 0.19301471 0.20036765 0.20036765 0.20772059 0.20772059
 0.21139706 0.21139706 0.22058824 0.22058824 0.23161765 0.23161765
 0.23713235 0.23713235 0.24080882 0.24080882 0.24816176 0.24816176
 0.25       0.25       0.25551471 0.25551471 0.26102941 0.26102941
 0.26286765 0.26286765 0.26470588 0.26470588 0.27205882 0.27205882
 0.27389706 0.27389706 0.27573529 0.27573529 0.27757353 0.27757353
 0.28308824 0.28308824 0.28492647 0.28492647 0.28860294 0.28860294
 0.29044118 0.29044118 0.29779412 0.29779412 0.29963235 0.29963235
 0.30147059 0.30147059 0.30330882 0.30330882 0.30698529 0.30698529
 0.31801471 0.31801471 0.32169118 0.32169118 0.32536765 0.32536765
 0.32720588 0.32720588 0.33272059 0.33272059 0.33455882 0.33455882
 0.33639706 0.33639706 0.34375    0.34375    0.35110294 0.35110294
 0.35294118 0.35294118 0.35477941 0.35477941 0.36397059 0.36397059
 0.36764706 0.36764706 0.36948529 0.36948529 0.37316176 0.37316176
 0.375      0.375      0.37683824 0.37683824 0.38051471 0.38051471
 0.38419118 0.38419118 0.38970588 0.38970588 0.39338235 0.39338235
 0.39522059 0.39522059 0.39889706 0.39889706 0.40257353 0.40257353
 0.40992647 0.40992647 0.41544118 0.41544118 0.41727941 0.41727941
```

```
0.41911765 0.41911765 0.42463235 0.42463235 0.42830882 0.42830882
0.43382353 0.43382353 0.4375     0.4375     0.44852941 0.44852941
0.45036765 0.45036765 0.45220588 0.45220588 0.45772059 0.45772059
0.46323529 0.46323529 0.47058824 0.47058824 0.48345588 0.48345588
0.48529412 0.48529412 0.48713235 0.48713235 0.49080882 0.49080882
0.49264706 0.49264706 0.49632353 0.49632353 0.5        0.5
0.50183824 0.50183824 0.50367647 0.50367647 0.50551471 0.50551471
0.50919118 0.50919118 0.52022059 0.52022059 0.52205882 0.52205882
0.52757353 0.52757353 0.52941176 0.52941176 0.54044118 0.54044118
0.54779412 0.54779412 0.54963235 0.54963235 0.55330882 0.55330882
0.5625     0.5625     0.56801471 0.56801471 0.56985294 0.56985294
0.57536765 0.57536765 0.57720588 0.57720588 0.58088235 0.58088235
0.58272059 0.58272059 0.58455882 0.58455882 0.59007353 0.59007353
0.59191176 0.59191176 0.59375    0.59375    0.60110294 0.60110294
0.60294118 0.60294118 0.61397059 0.61397059 0.61764706 0.61764706
0.61948529 0.61948529 0.62316176 0.62316176 0.62867647 0.62867647
0.63051471 0.63051471 0.63602941 0.63602941 0.64154412 0.64154412
0.64522059 0.64522059 0.65257353 0.65257353 0.65992647 0.65992647
0.66360294 0.66360294 0.66544118 0.66544118 0.66727941 0.66727941
0.67095588 0.67095588 0.67463235 0.67463235 0.67830882 0.67830882
0.68014706 0.68014706 0.68566176 0.68566176 0.68933824 0.68933824
0.69117647 0.69117647 0.70220588 0.70220588 0.70588235 0.70588235
0.70772059 0.70772059 0.70955882 0.70955882 0.71323529 0.71323529
0.71507353 0.71507353 0.71691176 0.71691176 0.72794118 0.72794118
0.73529412 0.73529412 0.73713235 0.73713235 0.73897059 0.73897059
0.74080882 0.74080882 0.74448529 0.74448529 0.74816176 0.74816176
0.75367647 0.75367647 0.75919118 0.75919118 0.77573529 0.77573529
0.77757353 0.77757353 0.78125    0.78125    0.79227941 0.79227941
0.79595588 0.79595588 0.79779412 0.79779412 0.79963235 0.79963235
0.80147059 0.80147059 0.80698529 0.80698529 0.81066176 0.81066176
0.81985294 0.81985294 0.82169118 0.82169118 0.82536765 0.82536765
0.82904412 0.82904412 0.83088235 0.83088235 0.83455882 0.83455882
0.83639706 0.83639706 0.84007353 0.84007353 0.84375    0.84375
0.84558824 0.84558824 0.84742647 0.84742647 0.85110294 0.85110294
0.85294118 0.85294118 0.85477941 0.85477941 0.85661765 0.85661765
0.86213235 0.86213235 0.86580882 0.86580882 0.86948529 0.86948529
0.87132353 0.87132353 0.87316176 0.87316176 0.88051471 0.88051471
0.88235294 0.88235294 0.88602941 0.88602941 0.88786765 0.88786765
0.89154412 0.89154412 0.89338235 0.89338235 0.89522059 0.89522059
0.89705882 0.89705882 0.89889706 0.89889706 0.90257353 0.90257353
0.90441176 0.90441176 0.90625    0.90625    0.90808824 0.90808824
0.90992647 0.90992647 0.91727941 0.91727941 0.92279412 0.92279412
0.92463235 0.92463235 0.92647059 0.92647059 0.92830882 0.92830882
0.93933824 0.93933824 0.94301471 0.94301471 0.94485294 0.94485294
0.95036765 0.95036765 0.95220588 0.95220588 0.95588235 0.95588235
0.95955882 0.95955882 0.96507353 0.96507353 0.96875    0.96875
0.97426471 0.97426471 0.98897059 0.98897059 0.99080882 0.99080882
0.99264706 0.99264706 0.99448529 0.99448529 1.         1.        ]
```

The ROC arrays generated here track the True Positive Rate against the False Positive Rate.

[13]:
```
y_test_pred_prob = logit.predict(sm.add_constant(X_test))
y_test_pred_prob
```

[13]:
```
985     0.390031
986     0.608480
987     0.448940
988     0.410107
989     0.433660
          …
1084    0.505413
1085    0.415704
1086    0.511403
1087    0.487963
1088    0.489563
Length: 104, dtype: float64
```

[14]:
```
metric = tpr * (1 - fpr)
optimal_idx = np.argmax(metric)
optimal_threshold = thresholds[optimal_idx]
print(f"Optimal Threshold: {optimal_threshold:.4f}")
```

```
Optimal Threshold: 0.5527
```

[15]:
```
y_test_pred_class = np.where(y_test_pred_prob >= optimal_threshold, 1, 0)
```

[16]:
```
cm = confusion_matrix(Y_test, y_test_pred_class)
cm
```

[16]:
```
array([[39,  4],
       [55,  6]])
```

The optimal threshold calculated to maximize $TPR(1 - FPR)$ is 0.5527.

When applying this threshold to the test data $(2009 - 2010)$, the model struggles. The confusion matrix shows it got 39 True Negatives and 6 True Positives, but a massive 55 False Negatives and 4 False Positives.

The resulting test error rate is 56.73%. This means the model is wrong more than half the time when using all predictors and this specific cutpoint.

f. Is there any change in the test error rate if we have worked with only the two predictors lag 1 and lag 2?

[17]:
```
test_error_rate = 1 - (np.trace(cm) / np.sum(cm))
test_error_rate
```

[17]:
```
np.float64(0.5673076923076923)
```

[18]:
```
predictors_f = ['Lag1', 'Lag2']
X_train_f = X_train[predictors_f]
X_train_f_const = sm.add_constant(X_train_f)
X_test_f =X_test[predictors_f]
X_test_f_const = sm.add_constant(X_test_f)
```

[19]:
```
logit_f = sm.Logit(Y_train, X_train_f_const).fit(disp=False)
test_probs_f = logit_f.predict(X_test_f_const)
Y_test_pred_f = (test_probs_f >= 0.5).astype(int)
```

[20]:
```
cm_f = confusion_matrix(Y_test, Y_test_pred_f)
test_error_f = 1 - (np.trace(cm_f) / np.sum(cm_f))
print(cm_f)
print(test_error_f)
```

```
[[ 7 36]
 [ 8 53]]
0.42307692307692313
```

Significant Improvement: Based on our calculations, yes there is a massive change. By dropping the non-informative predictors ($Lag3, Lag4, Lag5, Volume$) and relying only on $Lag1$ and $Lag2$, the test error rate dropped from 56.73% down to 42.3%.

Why this happens: Removing useless variables reduces the model's variance and prevents it from overfitting to the noise in the training data. The new confusion matrix shows it is much better at predicting the **"Up"** days (53 True Positives).